

Game Of Coins DP

1. Big Picture

This lesson is about the **Geeks and its Game of Coins** problem, which is a classic **Dynamic Programming (DP) + Game Theory** problem.

The game works like this:

- There are n coins.
- Two players play alternately.
- Player A (Geek) always moves first.
- On each turn, a player can remove:
 - 1 coin
 - x coins
 - y coins
- The player who removes the **last coin wins**.
- Both players play **optimally**.
- We need to determine whether Geek, the first player, can force a win.

The transcript's core idea is:

Instead of trying to simulate every possible game sequence, determine whether a particular number of remaining coins is a **winning state** or a **losing state**.

The instructor uses DP where:

```
dp[i] = 1 → the player whose turn it is can win with i coins
dp[i] = 0 → the player whose turn it is will lose with i coins
```

This is the key idea that makes the entire problem simple.

The transcript explains the problem, builds the DP states from 0 upward, derives the transition using moves 1, x , and y , and finally implements the solution.

2. Prerequisites

You only need a few concepts.

2.1 Dynamic Programming

Dynamic Programming means:

Solve smaller problems first, store their answers, and use those answers to solve larger problems.

For example:

```
dp[0]
dp[1]
dp[2]
dp[3]
...
dp[n]
```

Once we know the answer for smaller numbers of coins, we can determine the answer for a larger number.

2.2 Recursion / State Thinking

You should be comfortable thinking:

"If I make this move, what state will my opponent receive?"

For example, with 10 coins:

```
Take 1 → opponent gets 9
Take x → opponent gets 10-x
Take y → opponent gets 10-y
```

Then we ask:

Is any of those resulting states bad for my opponent?

If yes, we can force a win.

2.3 Basic Game Theory

There are two kinds of states:

Winning state

A state is **winning** if the current player has at least one move that makes the opponent enter a losing state.

Losing state

A state is **losing** if every legal move gives the opponent a winning state.

This distinction is the foundation of the solution.

3. Detailed Explanation

3.1 Understanding the Game

Suppose:

```
n = 5
X = 3
Y = 4
```

The legal moves are:

```
1 coin
3 coins
4 coins
```

Geek moves first.

Suppose Geek takes 3 :

```
5 → 2
```

Now the opponent has 2 coins.

The opponent cannot take 3 or 4, so they must take 1 :

```
2 → 1
```

Geek then takes the final coin:

```
1 → 0
```

Geek takes the last coin, so Geek wins.

Therefore:

```
answer = 1
```

The transcript uses this example to establish what the output means: 1 means Geek can win, while 0 means Geek cannot force a win.

3.2 The Most Important Mental Model

Forget the actual identities of Geek and his friend for a moment.

Instead, think:

```
dp[i] = Can the CURRENT PLAYER win if there are i coins?
```

This is extremely important.

We are **not** storing:

```
dp[i] = Can Geek win?
```

because after Geek makes a move, it becomes the opponent's turn.

So `dp[i]` should represent the player **whose turn it currently is**.

That makes the recurrence much easier.

3.3 What Does `dp[0]` Mean?

Suppose there are:

```
0 coins
```

and it is your turn.

Can you win?

No.

There is nothing to take.

So:

```
dp[0] = 0
```

This means:

```
A player facing 0 coins is in a losing state.
```

This is the fundamental base case.

3.4 What Does `dp[1]` Mean?

Suppose there is:

```
1 coin
```

and it is your turn.

You can take 1:

```
1 → 0
```

You took the last coin.

Therefore:

```
dp[1] = 1
```

So:

```
dp[0] = 0  
dp[1] = 1
```

3.5 What About `dp[2]`?

Suppose:

```
n = 2  
X = 3  
Y = 4
```

You cannot take 3.

You cannot take 4.

You can only take 1.

So:

```
2 → 1
```

Now the opponent has 1 coin.

The opponent takes it:

```
1 → 0
```

Therefore you lose.

So:

```
dp[2] = 0
```

Notice something important:

```
dp[2] = 0
```

doesn't mean Geek specifically loses.

It means:

```
Whoever is currently playing with 2 coins loses if both players play optimally.
```

3.6 `dp[3]`

Now:

```
n = 3
```

You can take 3.

Therefore:

3 → 0

You take the last coin.

So:

$dp[3] = 1$

3.7 dp[4]

You can take 4.

Therefore:

4 → 0

So:

$dp[4] = 1$

3.8 dp[5]

Now things become interesting.

Possible moves:

Take 1

5 → 4

We know:

$dp[4] = 1$

So the opponent gets a winning state.

Bad move.

Take 3

5 → 2

We know:

$dp[2] = 0$

That means the opponent gets a losing state.

Therefore taking 3 is a winning move.

So:

$dp[5] = 1$

Take 4

5 → 1

And:

$dp[1] = 1$

So that move is bad.

But we only need **one winning move**.

Taking 3 works.

Therefore:

$dp[5] = 1$

3.9 The Fundamental DP Rule

We can now derive the entire algorithm.

Suppose we are calculating:

$dp[i]$

The current player has three possible moves:

```
1
X
Y
```

After taking them, the opponent gets:

```
i - 1
i - X
i - Y
```

If **any one** of those states is losing for the opponent, the current player wins.

Therefore:

```
dp[i] = 1
```

if:

```
dp[i - 1] == 0
```

or:

```
dp[i - X] == 0
```

or:

```
dp[i - Y] == 0
```

provided those moves are actually possible.

Otherwise:

```
dp[i] = 0
```

3.10 Why Does This Logic Work?

This is the most important reasoning in the problem.

Suppose:

```
dp[5] = 1
```

because:

```
dp[5 - 3] = dp[2] = 0
```

That means:

```
Current player
  ↓
takes 3
  ↓
Opponent gets 2
  ↓
dp[2] = 0
  ↓
Opponent eventually loses
```

So the current player has found a move that forces the opponent into a losing position.

That's why:

Winning state = at least one move to a losing state.

Conversely:

Losing state = every legal move goes to a winning state.

3.11 Complete DP Table for $X = 3$, $Y = 4$

Let's construct the table from the transcript's example.

i	dp[i]	Reason
0	0	No coin → lose
1	1	Take 1
2	0	Only take 1 → $dp[1] = 1$
3	1	Take 3
4	1	Take 4
5	1	Take 3 → $dp[2] = 0$
6	1	Take 4 → $dp[2] = 0$
7	0	All moves lead to winning states
8	1	Take 1 → $dp[7] = 0$

So:

```
dp = [0, 1, 0, 1, 1, 1, 1, 0, 1]
```

The important pattern is:

```
0 = losing
1 = winning
```

3.12 Why Is $dp[7] = 0$?

This is a particularly useful example.

Suppose there are 7 coins.

Possible moves:

Take 1

```
7 → 6
```

And:

```
dp[6] = 1
```

So opponent wins.

Take 3

```
7 → 4
```

And:

```
dp[4] = 1
```

Opponent wins.

Take 4

```
7 → 3
```

And:

```
dp[3] = 1
```

Opponent wins.

So every possible move gives the opponent a winning state.

Therefore:

```
dp[7] = 0
```

This is the definition of a losing state.

3.13 The Critical Game-Theory Insight

This problem becomes much easier when you stop thinking:

"What move should Geek make?"

and instead think:

"Can the current player move the opponent into a losing state?"

That gives the generic game DP pattern:

```
If I can move to a losing state:  
    I win.
```

```
If every move goes to a winning state:  
    I lose.
```

You will see this pattern in many game problems.

3.14 Formal DP Transition

Let:

```
dp[i] = 1
```

if the current player can force a win with i coins.

Then:

```
dp[i] = 1
```

if at least one of these is true:

```
i >= 1 && dp[i - 1] == 0
```

or:

```
i >= X && dp[i - X] == 0
```

or:

```
i >= Y && dp[i - Y] == 0
```

Otherwise:

```
dp[i] = 0
```

In compact mathematical form:

```
dp[i] = 1  
if  $\exists$  move  $m \in \{1, X, Y\}$  such that  $i \geq m$  and  $dp[i-m] = 0$   
  
dp[i] = 0  
otherwise
```

3.15 Why Do We Check $i \geq X$?

Suppose:

```
i = 2  
X = 3
```

You cannot take 3 coins when only 2 exist.

Therefore we must not access:

```
dp[2 - 3]
```

which would be:

```
dp[-1]
```

So before using X :

```
if (i >= X && dp[i - X] == 0)
```

Similarly for Y .

3.16 Implementation

A clean C++ implementation is:

```
class Solution {  
public:  
    int findWinner(int n, int x, int y) {  
        vector<int> dp(n + 1, 0);  
  
        dp[0] = 0;
```

```

for (int i = 1; i <= n; i++) {

    // Take 1 coin
    if (dp[i - 1] == 0) {
        dp[i] = 1;
    }

    // Take x coins
    else if (i >= x && dp[i - x] == 0) {
        dp[i] = 1;
    }

    // Take y coins
    else if (i >= y && dp[i - y] == 0) {
        dp[i] = 1;
    }

    // Otherwise dp[i] remains 0
}

return dp[n];
}
};

```

The exact function name may differ depending on the platform's required signature.

3.17 Code Walkthrough

Let's understand the code instead of memorizing it.

Step 1: Create DP array

```
vector<int> dp(n + 1, 0);
```

Why `n + 1`?

Because we need indexes:

```
0, 1, 2, ..., n
```

That's `n + 1` states.

Initially every state is set to `0`.

Step 2: Base state

```
dp[0] = 0;
```

With zero coins, the current player cannot move.

Therefore they lose.

Step 3: Iterate through states

```
for (int i = 1; i <= n; i++)
```

We calculate:

```

dp[1]
dp[2]
dp[3]
...
dp[n]

```

This works because every state depends only on smaller states.

Step 4: Try taking one coin

```
if (dp[i - 1] == 0)
```

If we take one coin, the opponent gets:

```
i - 1
```


coins.

If:

```
dp[i - 1] == 0
```

then the opponent is entering a losing state.

Therefore we can win:

```
dp[i] = 1;
```

Step 5: Try taking X

```
else if (i >= x && dp[i - x] == 0)
```

We can take x only if:

```
i >= X
```

After taking X:

```
remaining = i - X
```

If:

```
dp[i - X] == 0
```

the opponent receives a losing state.

Therefore:

```
dp[i] = 1;
```

Step 6: Try taking Y

Same logic:

```
else if (i >= y && dp[i - y] == 0)
```

If taking Y sends the opponent to a losing state, the current player wins.

3.18 Why else if Is Fine Here

You might wonder:

Why don't we need to check all three conditions?

We don't need to know **how many** winning moves exist.

We only need to know whether **at least one** exists.

For example:

```
Take 1 → opponent wins
Take X → opponent loses
Take Y → opponent wins
```

That's enough.

We found one winning move:

```
Take X
```

Therefore:

```
dp[i] = 1
```

3.19 Complete Dry Run

Let's use:

```
n = 8
X = 3
Y = 4
```

State 0

```
dp[0] = 0
```

State 1

Take 1:

1 → 0

dp[0] = 0, therefore:

dp[1] = 1

State 2

Take 1:

2 → 1

But:

dp[1] = 1

So opponent wins.

Cannot take 3 or 4.

Therefore:

dp[2] = 0

State 3

Take 3:

3 → 0

Therefore:

dp[3] = 1

State 4

Take 4:

4 → 0

Therefore:

dp[4] = 1

State 5

Try 1:

5 → 4

dp[4] = 1.

Not good.

Try 3:

5 → 2

dp[2] = 0.

Good.

Therefore:

dp[5] = 1

State 6

Try 1:

6 → 5

dp[5] = 1.

Try 3:

6 → 3

$dp[3] = 1$.

Try 4:

$6 \rightarrow 2$

$dp[2] = 0$.

Therefore:

$dp[6] = 1$

State 7

Try 1:

$7 \rightarrow 6$
 $dp[6] = 1$

Bad.

Try 3:

$7 \rightarrow 4$
 $dp[4] = 1$

Bad.

Try 4:

$7 \rightarrow 3$
 $dp[3] = 1$

Bad.

There is no winning move.

Therefore:

$dp[7] = 0$

State 8

Try 1:

$8 \rightarrow 7$

And:

$dp[7] = 0$

Excellent.

Therefore:

$dp[8] = 1$

We don't even need to test the other moves.

3.20 Final DP Table

For:

$X = 3$
 $Y = 4$

we get:

Coins i	$dp[i]$	Reason
0	0	No move
1	1	Take 1
2	0	Only move to winning state
3	1	Take 3
4	1	Take 4
5	1	Take 3 $\rightarrow$ state 2
6	1	Take 4 $\rightarrow$ state 2
7	0	Every move $\rightarrow$ winning state
8	1	Take 1 $\rightarrow$ state 7

So:

```
dp = [0,1,0,1,1,1,1,0,1]
```

3.21 Time Complexity

We calculate every state once:

```
0 → n
```

For each state, we check at most three moves:

```
1  
X  
Y
```

Therefore:

```
Time Complexity = O(n)
```

The constant 3 does not matter in Big-O notation.

3.22 Space Complexity

We store:

```
dp[0 ... n]
```

Therefore:

```
Space Complexity = O(n)
```

3.23 Can We Optimize the Space?

Yes, but there is an important point.

The recurrence needs:

```
dp[i - 1]  
dp[i - X]  
dp[i - Y]
```

So we may need values potentially far behind the current index.

Therefore, unlike Fibonacci where only the previous two values are required, a simple constant-space optimization is generally **not available** when X and Y can be arbitrary.

The $O(n)$ DP array is the straightforward implementation.

3.24 A Better Way to Think About the Entire Problem

Whenever you see a game problem like this, ask these four questions:

Question 1

What is my state?

```
Number of remaining coins
```

Question 2

What are my possible moves?

```
1, X, Y
```

Question 3

What happens after I make a move?

```
i → i-1  
i → i-X  
i → i-Y
```

Question 4

When can I win?

If I can move the opponent into a losing state:

```
dp[next] == 0
```

That's the whole algorithm.

4. Key Takeaways

1. DP state

```
dp[i]
```

means:

```
Can the player whose turn it currently is force a win with i coins?
```

2. Base case

```
dp[0] = 0;
```

No coins means the current player cannot move and therefore loses.

3. Winning condition

If **any** legal move reaches a losing state:

```
dp[next] == 0
```

then:

```
dp[i] = 1;
```

4. Losing condition

If **all** legal moves reach winning states:

```
dp[next] == 1
```

then:

```
dp[i] = 0;
```

5. The most important game-DP rule

Memorize the idea, not the code:

```
Winning state
  ↓
There exists a move
  ↓
Opponent enters losing state
```

while:

```
Losing state
  ↓
Every possible move
  ↓
Opponent enters winning state
```

6. Complexity

```
Time = O(n)
Space = O(n)
```

5. Common Mistakes / Misunderstandings

Mistake 1: Thinking `dp[i]` means "Can Geek win?"

That's the biggest conceptual mistake.

It means:

```
Can the current player win?
```

This automatically handles the fact that turns alternate.

Mistake 2: Thinking `dp[i] = 1` means the current player wins immediately

No.

It means the current player has a strategy that eventually guarantees a win, assuming the opponent also plays optimally.

Mistake 3: Forgetting the opponent's perspective

Suppose:

```
dp[5] = 1
```

because:

```
5 → 2
```

and:

```
dp[2] = 0
```

The reason 5 is winning is not that 2 is good.

It's exactly the opposite:

```
2 is bad for the next player.
```

Mistake 4: Allowing impossible moves

For:

```
i = 2  
X = 3
```

you cannot do:

```
i - X
```

without checking whether `i >= X`.

Use:

```
if (i >= x && dp[i - x] == 0)
```

Mistake 5: Using `dp[1] = 0`

With one coin:

```
1 → 0
```

The current player takes the last coin.

Therefore:

```
dp[1] = 1
```

Mistake 6: Simulating the entire game tree

You could recursively try every possible sequence:

```
take 1  
take X  
take Y
```

and continue recursively.

But many states would be recomputed.

For example, the state:

```
remaining = 5
```

could be reached through many different sequences.

DP solves that by calculating each state once.

Mistake 7: Confusing "there exists" with "all"

For a winning state:

```
ANY winning move is enough.
```

For a losing state:

```
ALL moves must be bad.
```

This distinction appears in many two-player game DP problems.

6. Quick Revision

Problem

Given:

```
n coins  
moves = {1, X, Y}
```

Geek moves first.

The player taking the last coin wins.

Determine whether Geek wins.

DP Definition

```
dp[i] = 1
```

if the current player can force a win with i coins.

```
dp[i] = 0
```

if the current player will lose.

Base Case

```
dp[0] = 0;
```

Transition

```
if dp[i-1] == 0  
    dp[i] = 1  
  
else if i >= X && dp[i-X] == 0  
    dp[i] = 1  
  
else if i >= Y && dp[i-Y] == 0  
    dp[i] = 1  
  
else  
    dp[i] = 0
```

Final Answer

```
return dp[n];
```

Complexity

```
O(n) time  
O(n) space
```

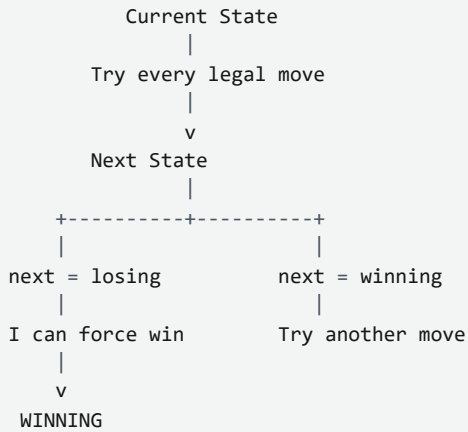
7. Questions You Should Be Able to Answer

Before considering this problem understood, you should be able to answer these without looking at the solution:

1. What does $dp[i]$ represent?
2. Why is $dp[0] = 0$?
3. Why is $dp[1] = 1$?
4. For $n = 5$, $X = 3$, $Y = 4$, why is $dp[5] = 1$?
5. Why is $dp[7] = 0$ for $X = 3$, $Y = 4$?
6. Why do we check $dp[i - X] == 0$?
7. Why does finding **one** losing next state make the current state winning?
8. Why does every possible move leading to a winning state make the current state losing?
9. Why must we check $i \geq X$ before accessing $dp[i-X]$?
10. Why does $dp[i]$ represent the **current player**, rather than specifically Geek?

11. What would happen if players were allowed to take 2 coins as well?
12. What is the time complexity?
13. Why is the basic solution $O(n)$ space?
14. Could you manually construct `dp[0...10]` for $X = 3$, $Y = 4$?
15. How would you recognize this as a **game-state DP** problem in an interview?

The most important thing to internalize is this single pattern:



Once this pattern becomes natural, a large class of **two-player optimal-game DP problems** becomes much easier to recognize and solve.